

$\Rightarrow$ BST

- * Searching in a BST
- * Insertion in a BST
- * how check if BT is a BST

boolean isBST(root, l, h) {

if (root == null)
return true;

if (root->data >= l &&
root->data <= h) {

boolean left = isBST(root->left, l, root->data-1);

boolean right = isBST(root->right, root->data+1, h);

return left && right;

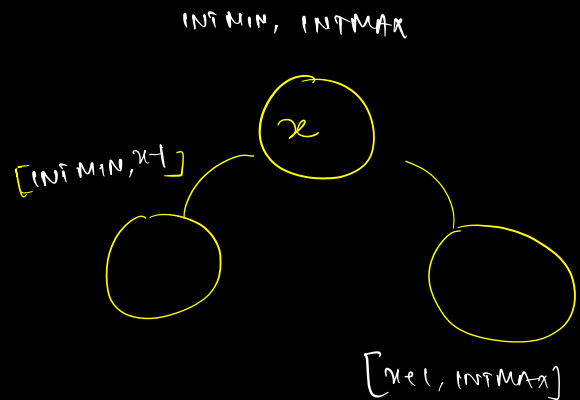
}

return false

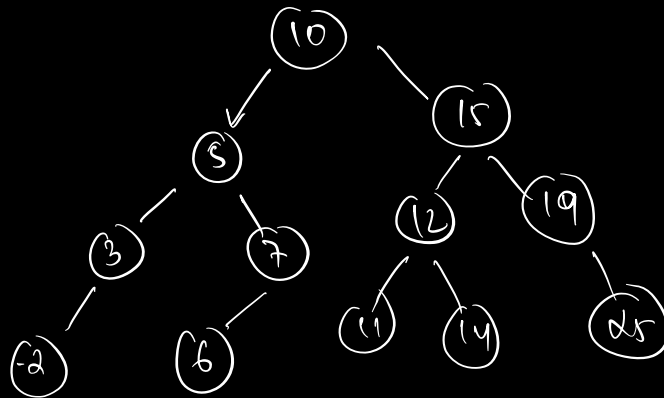
}

TC $\Rightarrow O(N)$
SC $\Rightarrow O(h)$

recursion stack



⇒ Deletion of a node in a Bst:

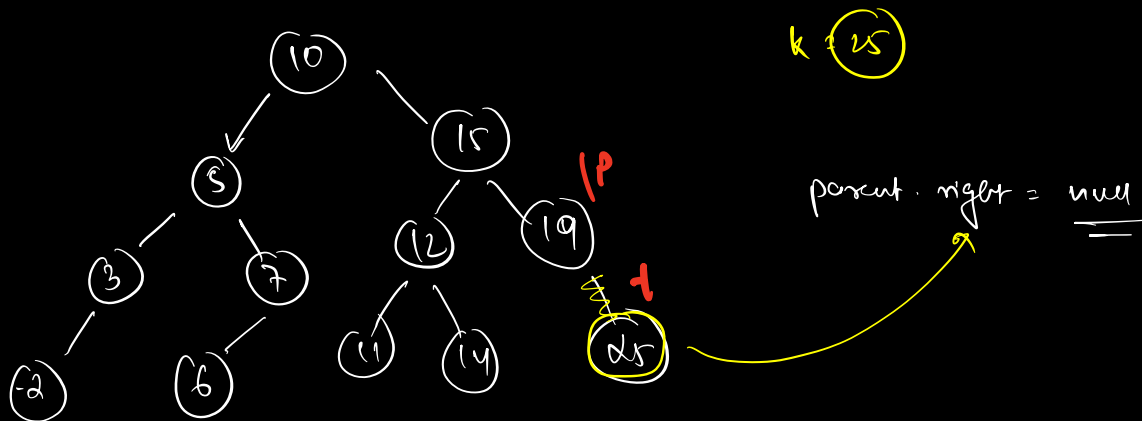


1) deleting a node with no child

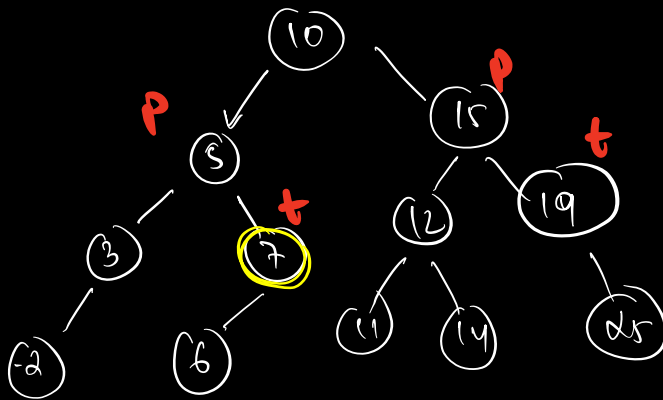
2) deleting a node with 1 child

3) deleting a node with 2 children

1) deleting a node with no child



" deleting a node with 1 child



k = 7

```

if ( parent.right == t ) {
    parent.right = t.right;
    if ( temp.left != null ) {
        parent.right = t.left;
    }
}

```

```

parent.right = t.right;
if ( temp.left != null ) {
    parent.right = temp.left;
}

```

```

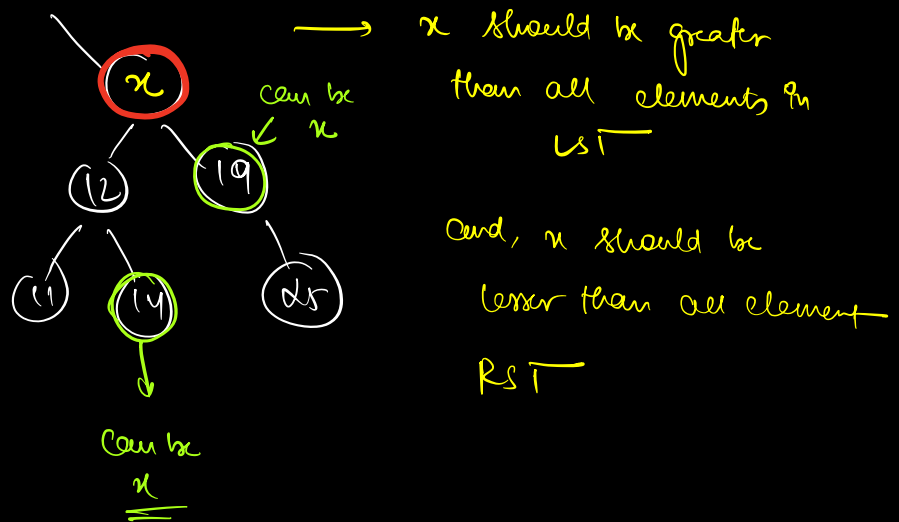
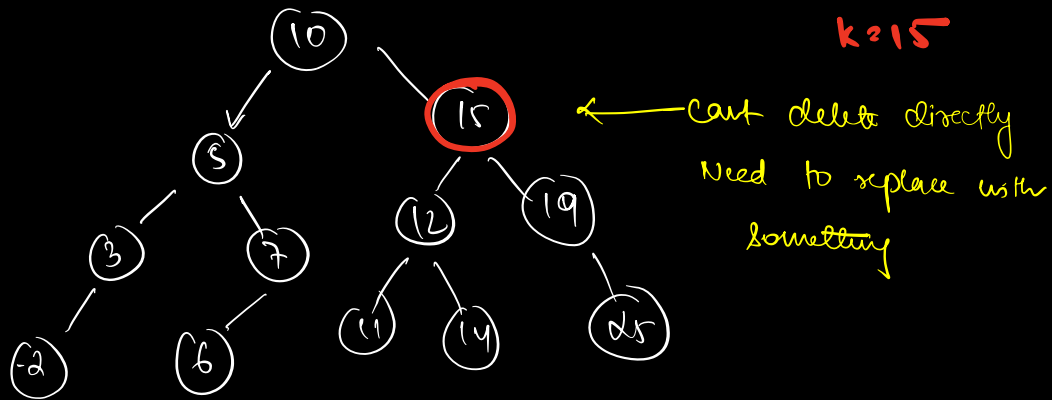
} else {
    parent.left = t.right;
    if ( temp.left != null ) {
        parent.left = t.left;
    }
}

```



will work for no child or 1 child

3) deleting a node with 2 children



* possible values of x , would be,

either last value of RST, or max^m value in LST.

min in a BST $\Rightarrow$ temp = root (prev)

while(temp.left != null) {

temp = temp.left;

}

$\rightarrow$ smallest $\Rightarrow$ temp.data

max in a BST $\Rightarrow$ temp = root | prev

```

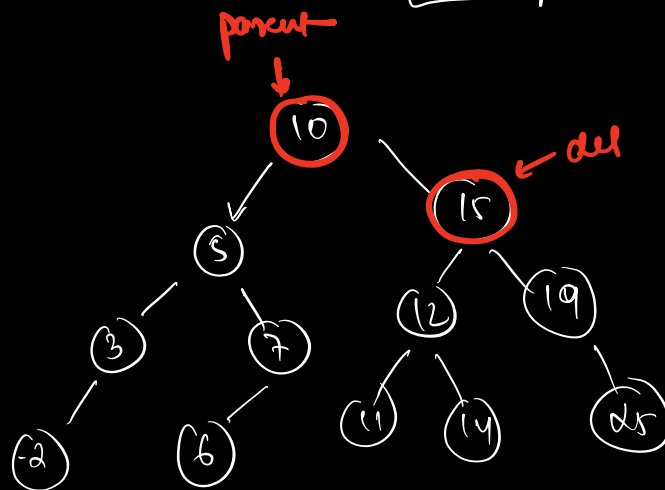
while (temp.right != null) {
    temp = temp.right
}

```

$\rightarrow$ max $\Rightarrow$ temp.data

1) as reach to the node that needs to be deleted $\Rightarrow$ del

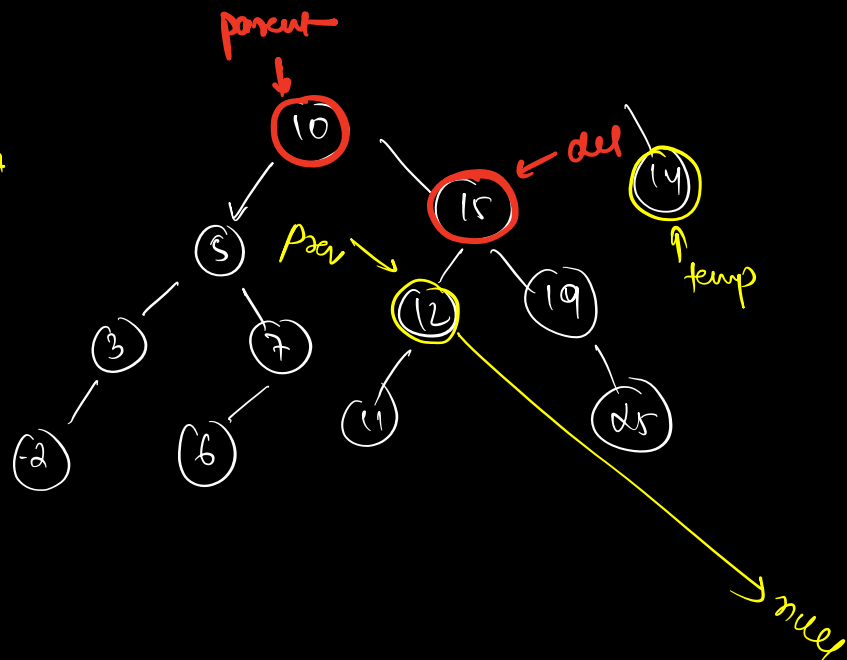
2) maintain a parent ptr, for the node that needs to be deleted $\Rightarrow$ parent



1) as find either, max of LST (left of del node)
 or, min of RST (right of del node)
 (temp)

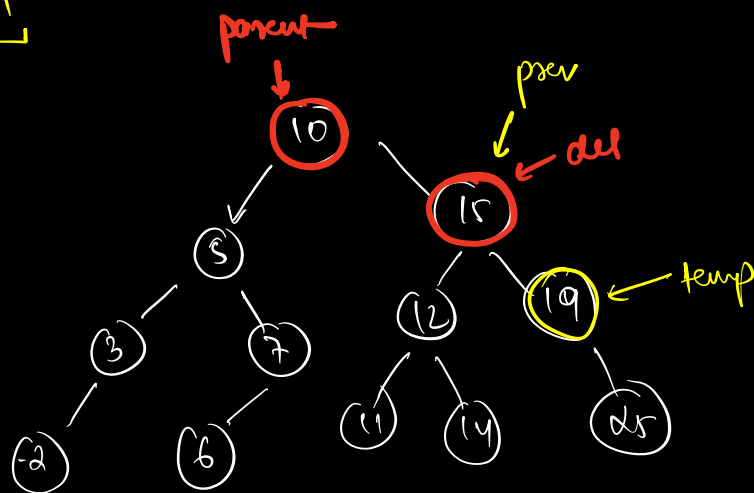
b) while searching for max(LST) or min(RST)
maintain a prev ptr.

i) max of LST



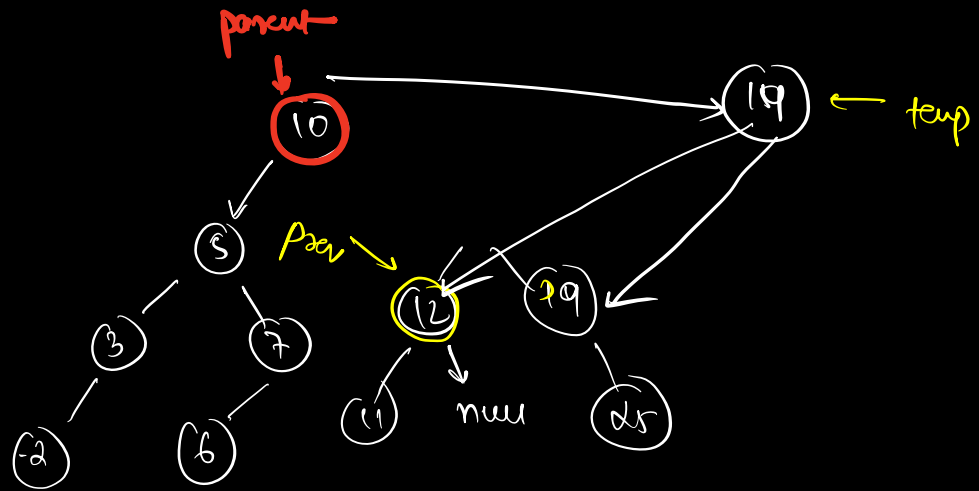
or,

ii) min of RST

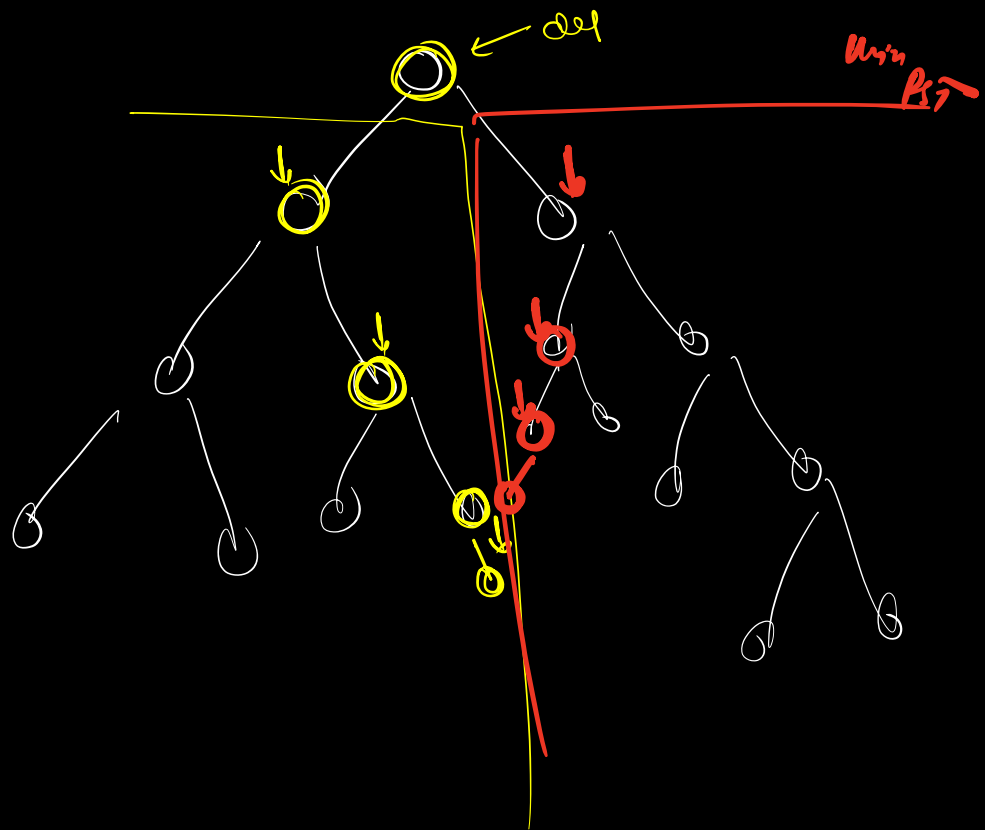


iii) delete (temp) $\Rightarrow$ [temp will always have 0/1 child]

iv) [parent.left/right = temp.]



max LST



⇒ replace → del with temp

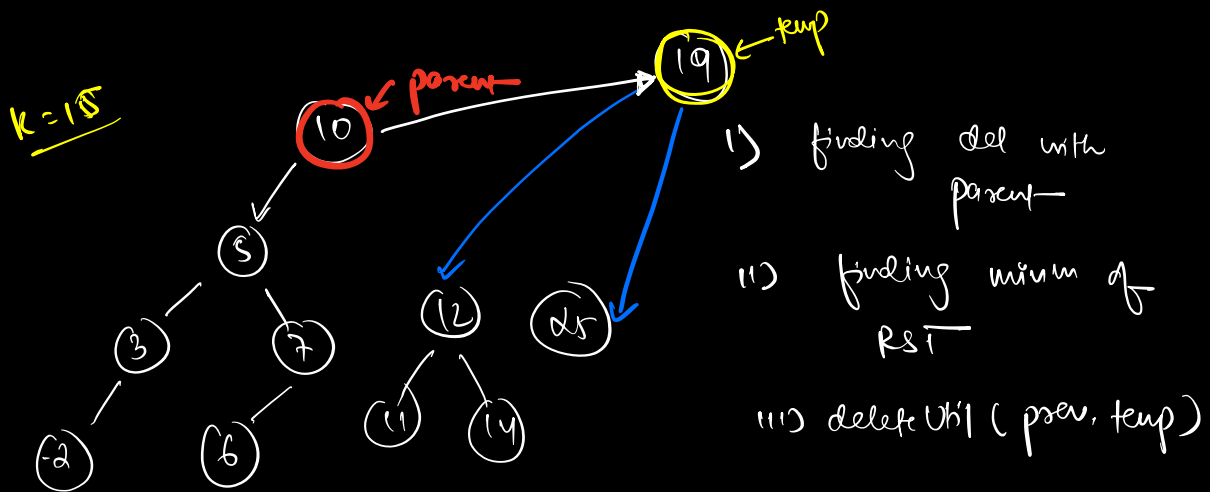
temp.right = del.right

temp.left = del.left

```

if (parent.left == del) {
    parent.left = temp;
} else {
    parent.right = temp;
}
}

```



```

void deleteUtil (parent, temp) {
    if (parent.right == t) {
        parent.right = t.right;
        if (temp.left != null) {
            parent.right = t.left;
        }
    } else {
        parent.left = t.right;
        if (temp.left != null) {
            parent.left = t.left;
        }
    }
}

```


3 5

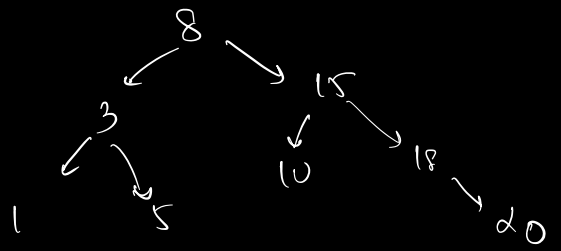
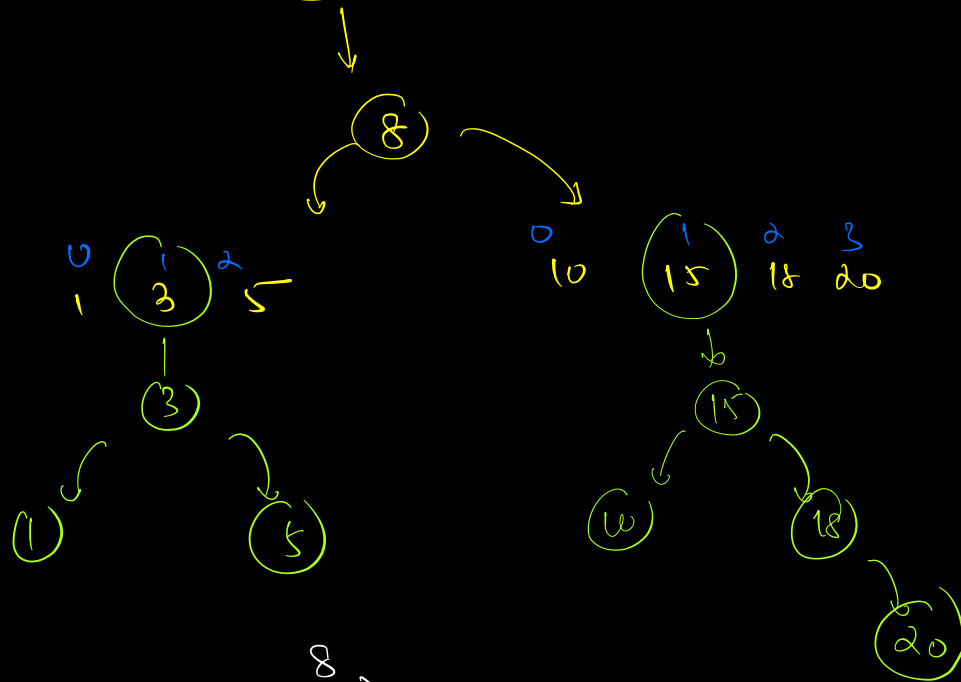
Steps

- 1) find the node that has to be deleted $\rightarrow$ del
 * with its parent $\rightarrow$ parent
- 2) find replacement $\rightarrow$ temp
 1) max of LST
 or 1) min of RST
 with their parent $\rightarrow$ prev
- iii) deleteNode (prev, temp)
- iv) replace the del node by temp node

$$\left\{ \begin{array}{l} TC \Rightarrow O(2h) \approx O(h) \\ SC \Rightarrow [\text{---}] \end{array} \right\}$$

: Build a BBST from a sorted array:-

arr $\Rightarrow$ $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 3 & 5 & 8 & 10 & 15 & 18 & 20 \end{matrix} \rightarrow \text{inorder}$



Node build(arr, l, r) {

if (l > r) return null;

int mid = $\frac{l + (r - l)}{2}$

Node n = new Node(mid);

n.left = build(arr, l, mid - 1)

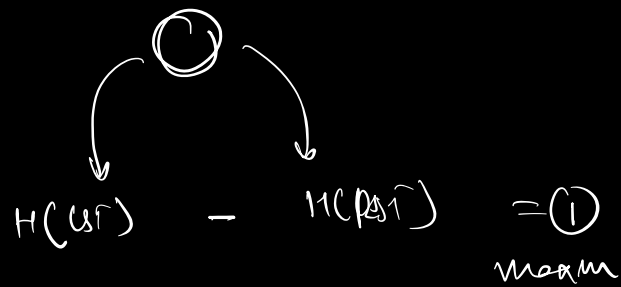
n. right = build (arr, mid+1, r)

return n

}

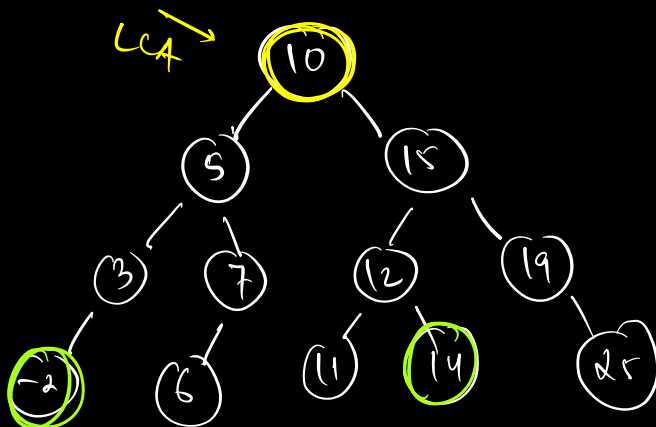
$TC \Rightarrow O(N)$
 $SC \Rightarrow O(h)$

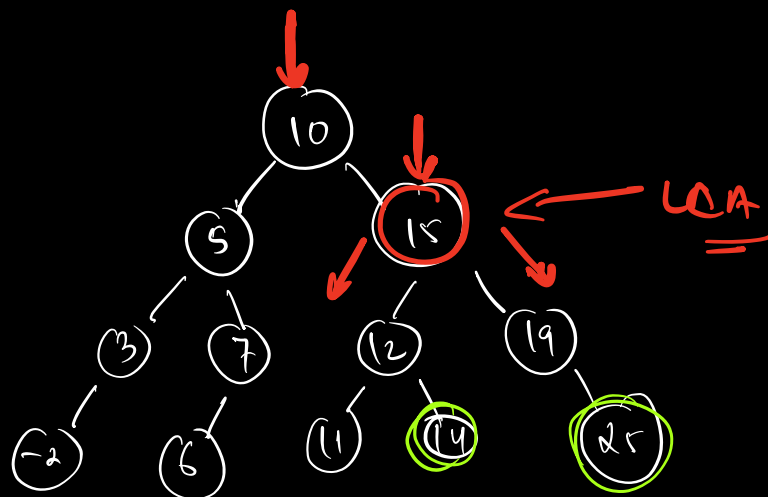
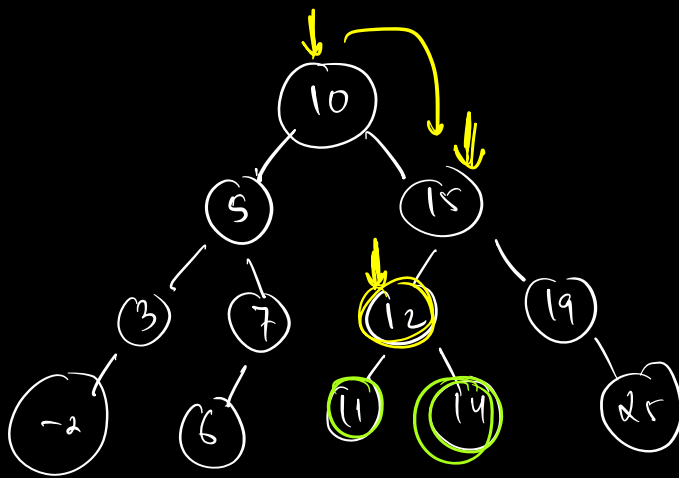
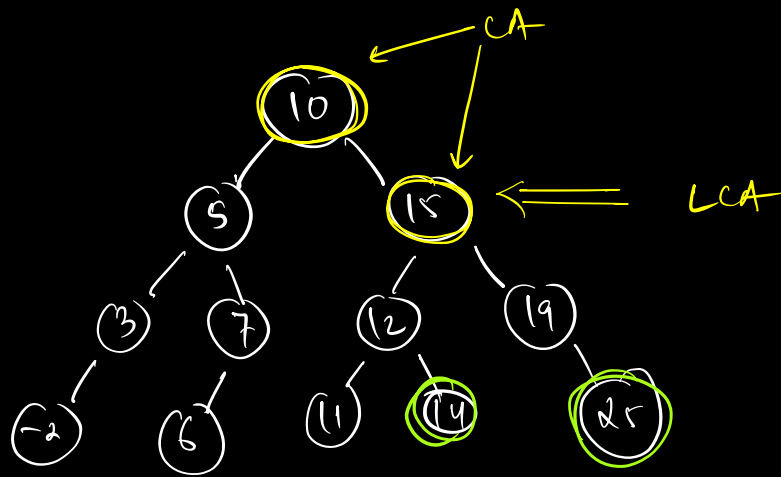
B^BST $\rightarrow$ balanced BST

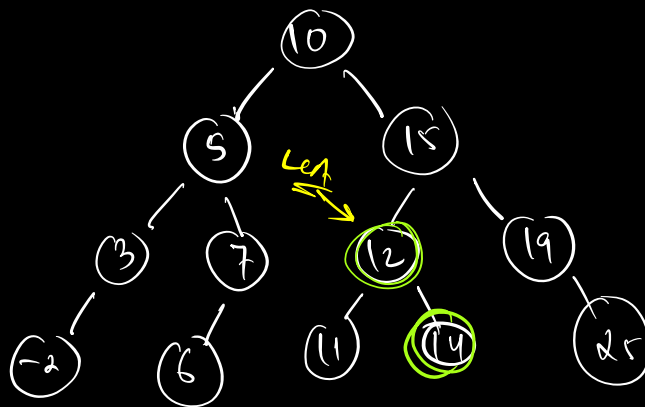


$\Rightarrow$ LCA of 2 nodes in a BST :-

$\downarrow$
lowest common ancestor





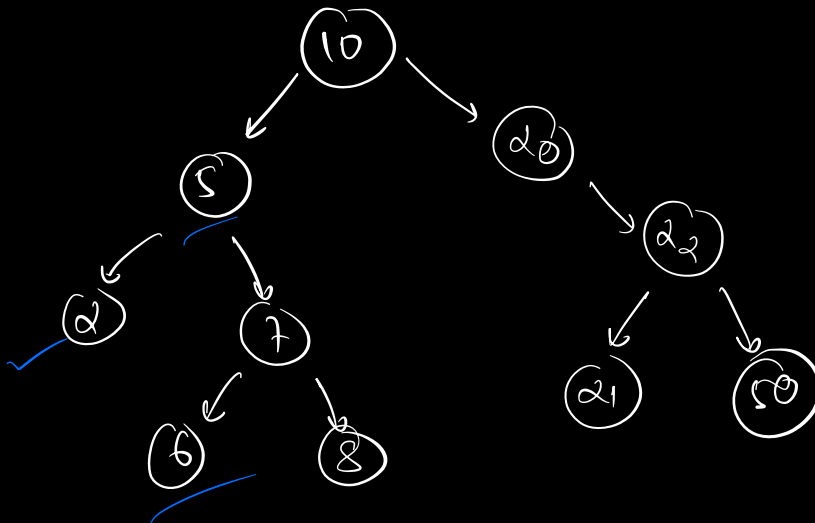


```

Node  LCABSF( root, x, y) {
    temp = root
    while (temp != null) {
        if ( (x.data < temp.data &&
              y.data < temp.data) ) {
            temp = temp.left;
        } else if ( (x.data > temp.data &&
                     y.data > temp.data) ) {
            temp = temp.right;
        }
        else {
            return temp;
        }
    }
}

```

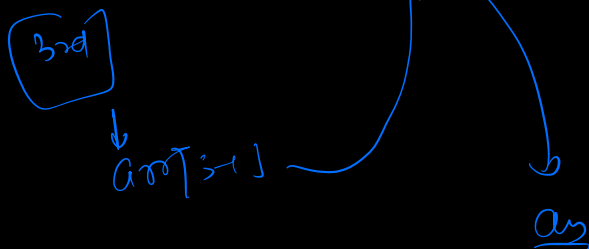
$\Rightarrow$ k^{th} smallest element in a BST.



3rd smallest $\Rightarrow$ 6

5th smallest $\Rightarrow$ 8

Inorder $\Rightarrow$ 2 5 6 7 8 10 20 22 50



TC $\Rightarrow$ $O(N)$
 SC $\Rightarrow$ $O(N)$ } Inorder

$\downarrow$
 SC $\Rightarrow$ $[O(1) \rightarrow O(1)]$ Is it
 doable?

↓
[Morris Inorder
traversal]

↓
next-class